Debugging a learning algorithm:

Suppose you have implemented regularized linear regression to predict housing prices.

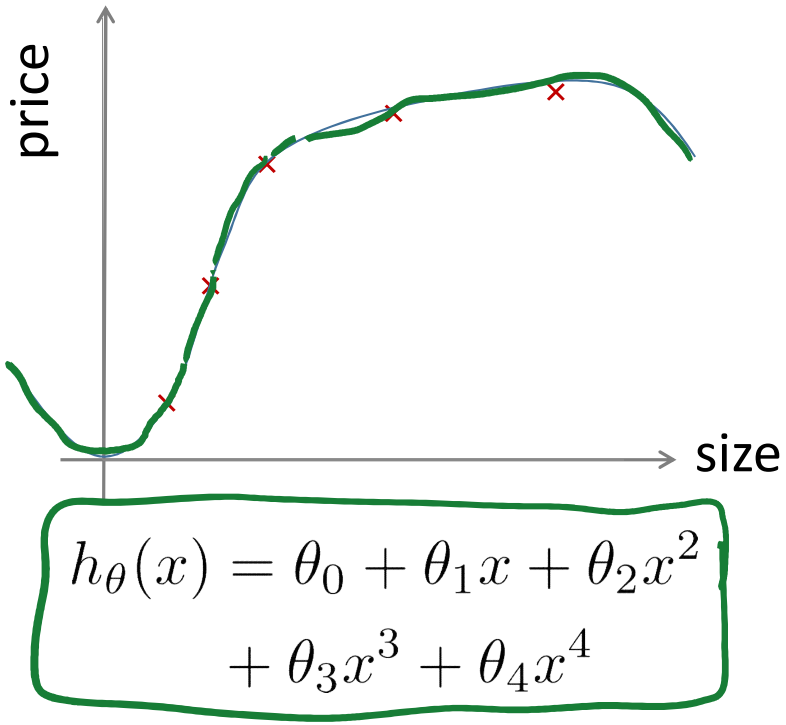
$$J(\theta) = \frac{1}{2m}\left[\sum_{i=1}^{m}(h_\theta(x^{(i)}) - y^{(i)})^2 + \lambda\sum_{j=1}^{m}\theta_j^2\right]$$

However, when you test your hypothesis on a new set of houses, you find that it makes unacceptably large errors in its predictions. What should you try next?

- Get more training examples
- Try smaller sets of features
- Try getting additional features
- Try adding polynomial features $(x_1^2, x_2^2, x_1x_2, \text{etc.})$
- Try decreasing λ
- Try increasing λ

Evaluating your hypothesis

overfitting Var ↑
underfitting Bias ↑



Fails to generalize to new examples not in training set.

$x_1 =$ size of house
$x_2 =$ no. of bedrooms
$x_3 =$ no. of floors
$x_4 =$ age of house
$x_5 =$ average income in neighborhood
$x_6 =$ kitchen size
$\vdots$
x_{100}

Evaluating your hypothesis

Dataset:

Size	Price
2104	400
1600	330
2400	369
1416	232
3000	540
1985	300
1534	315
1427	199
1380	212
1494	243

7 :

3 :

Training set

test set

$$(x^{(1)}, y^{(1)})$$
$$(x^{(2)}, y^{(2)})$$
$$\vdots$$
$$(x^{(m)}, y^{(m)})$$

$$(x_{test}^{(1)}, y_{test}^{(1)})$$
$$(x_{test}^{(2)}, y_{test}^{(2)})$$
$$\vdots$$
$$(x_{test}^{(m_{test})}, y_{test}^{(m_{test})})$$

Training/testing procedure for linear regression

m

- Learn parameter θ from training data (minimizing training error $J(\theta)$)

$\theta_{opt} = \min_{\theta} J_{train}(\theta)$

- Compute test set error:

$$J_{\text{test}}(\theta) = \frac{1}{2mtest} \sum_{i=1}^{mtest} \left(h_\theta\left(x_{\text{test}}^{(i)}\right) - y_{\text{test}}^{(i)} \right)^2$$

log Reg: $J_{test}(\theta) = -\frac{1}{m_{test}} \sum_{i=1}^{m_{test}} y_{test}^{(i)} \log h_\theta(x_{test}^{(i)}) + (1 - y_{test}^{(i)}) \log(1 - h_\theta(x_{test}^{(i)}))$

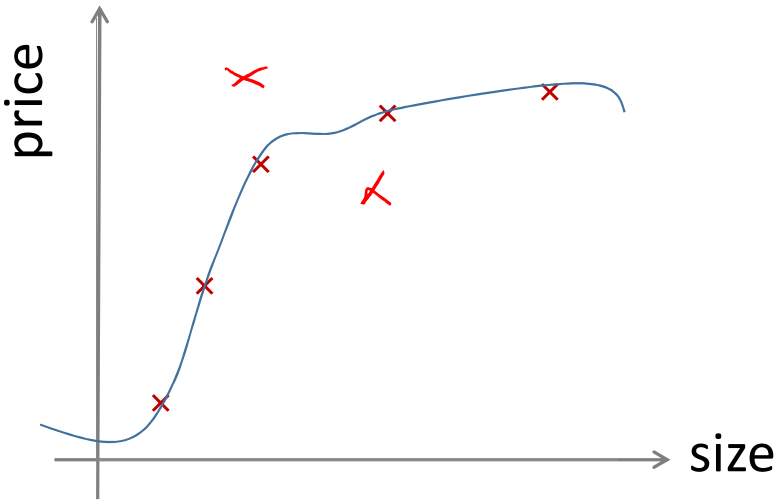
یا فلاً در مسائل طبقه‌بندی باینری خطای تست به صورت زیر تعریف می‌شود:

$$error(h_\theta(x), y) = \begin{cases} 1 & \text{if } \begin{cases} h_\theta(x) \geq 0.5 \;,\; y=0 \\ h_\theta(x) < 0.5 \;,\; y=1 \end{cases} \\ 0 & \text{o.w} \end{cases}$$

$$\text{Test error} = \frac{1}{m_{test}} \sum_{i=1}^{m_{test}} error\left(h_\theta(x^{(i)}_{test}), y^{(i)}_{test}\right)$$

خطای داده تست

Overfitting example



$$h_\theta(x) = \theta_0 + \theta_1 x + \theta_2 x^2 + \theta_3 x^3 + \theta_4 x^4$$

$J_{train} \simeq 0$

J_{test} ↑

Once parameters $\theta_0, \theta_1, \ldots, \theta_4$ were fit to some set of data (training set), the error of the parameters as measured on that data (the training error $J(\theta)$) is likely to be lower than the actual generalization error.

Model selection

d = 1 1. $h_\theta(x) = \theta_0 + \theta_1 x$

d = 2 2. $h_\theta(x) = \theta_0 + \theta_1 x + \theta_2 x^2$

d = 3 3. $h_\theta(x) = \theta_0 + \theta_1 x + \cdots + \theta_3 x^3$

$\vdots$

d = 10 10. $h_\theta(x) = \theta_0 + \theta_1 x + \cdots + \theta_{10} x^{10}$

Choose $\theta_0 + \ldots \theta_5 x^5$

How well does the model generalize? Report test set error $J_{test}(\theta^{(5)})$.

Problem: $J_{test}(\theta^{(5)})$ is likely to be an optimistic estimate of generalization error. I.e. our extra parameter (d = degree of polynomial) is fit to test set.

Evaluating your hypothesis

Dataset: $\{x^{(i)}, y^{(i)}\}$

Size	Price
2104	400
1600	330
2400	369
1416	232
3000	540
1985	300
1534	315
1427	199
1380	212
1494	243

60%

20%

20%

Training set

Cross validation set(CV)

Test set

$(x^{(1)}, y^{(1)})$
$(x^{(2)}, y^{(2)})$
$\vdots$
$(x^{(m)}, y^{(m)})$

$J(\theta)_{train}$

$(x_{cv}^{(1)}, y_{cv}^{(1)})$
$(x_{cv}^{(2)}, y_{cv}^{(2)})$
$\vdots$
$(x_{cv}^{(m_{cv})}, y_{cv}^{(m_{cv})})$

$J_{cv}(\theta)$

$(x_{test}^{(1)}, y_{test}^{(1)})$
$(x_{test}^{(2)}, y_{test}^{(2)})$
$\vdots$
$(x_{test}^{(m_{test})}, y_{test}^{(m_{test})})$

$J_{test}(\theta)$

Train/validation/test error

Training error:

$$J_{train}(\theta) = \frac{1}{2m} \sum_{i=1}^{m} (h_\theta(x^{(i)}) - y^{(i)})^2$$

MSE

Cross Validation error:

$$J_{cv}(\theta) = \frac{1}{2m_{cv}} \sum_{i=1}^{m_{cv}} (h_\theta(x_{cv}^{(i)}) - y_{cv}^{(i)})^2$$

Test error:

$$J_{test}(\theta) = \frac{1}{2m_{test}} \sum_{i=1}^{m_{test}} (h_\theta(x_{test}^{(i)}) - y_{test}^{(i)})^2$$

Model selection

1000 : 600 train (200 CV) 200 test

1. $h_\theta(x) = \theta_0 + \theta_1 x$ (d=1) $\Rightarrow$ argmin J_{train} $\Rightarrow \theta^{(1)} \Rightarrow J_{CV}(\theta^{(1)})$
2. $h_\theta(x) = \theta_0 + \theta_1 x + \theta_2 x^2$ (d=2)
3. $h_\theta(x) = \theta_0 + \theta_1 x + \cdots + \theta_3 x^3$

$\vdots$

10. $h_\theta(x) = \theta_0 + \theta_1 x + \cdots + \theta_{10} x^{10}$ (d=10) $\Rightarrow \theta^{(10)} \Rightarrow J_{CV}(\theta^{(10)})$

$d_{opt} = \arg\min_d J_{CV}(\theta^{(d)})$

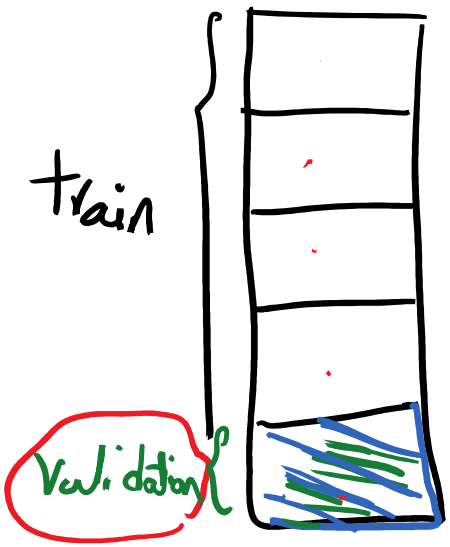
Pick $\theta_0 + \theta_1 x_1 + \cdots + \theta_4 x^4$ $d_{opt} = 4$

Estimate generalization error for test set $J_{test}(\theta^{(4)})$

$J_{test}(\theta^{(d_{opt})})$

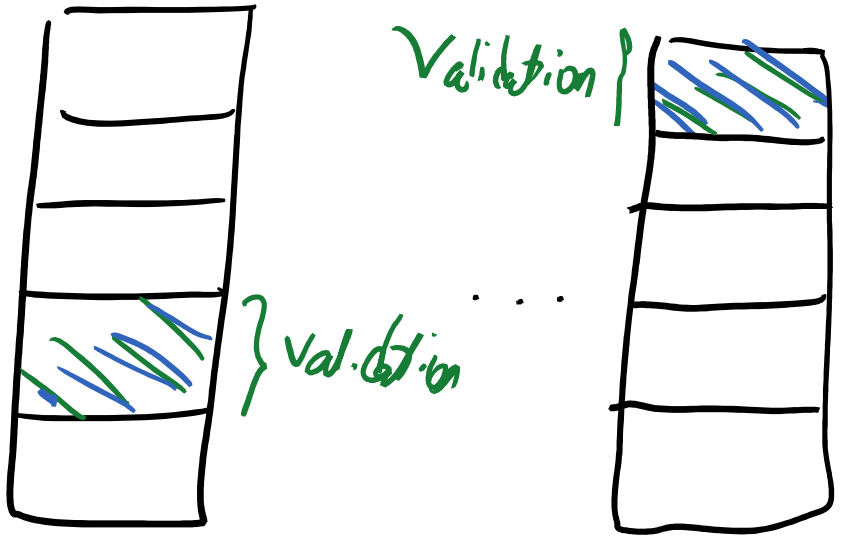
K_Fold cross validation.

5_Fold



J_{cv} روی 5 مقدار

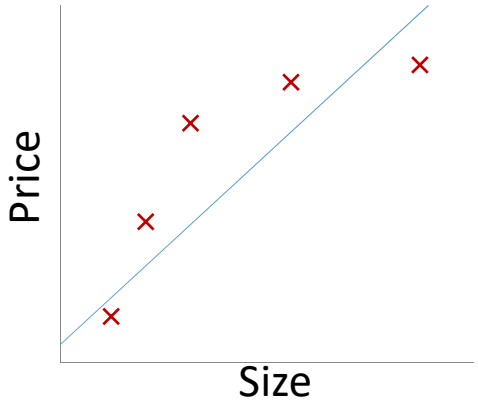
متوسط گرفته می‌شود.



$k = m$ → leave_one_out

(Validation هر بار فقط 1 نمونه)

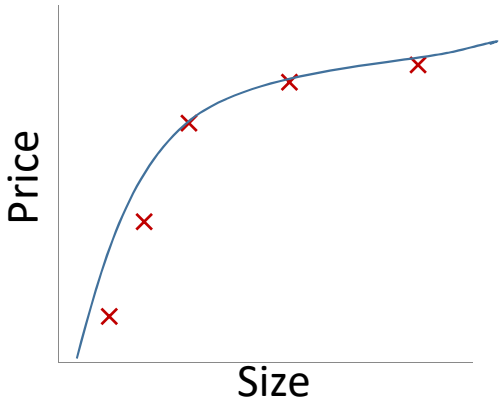
Bias/variance



$\theta_0 + \theta_1 x$

High bias (underfit)

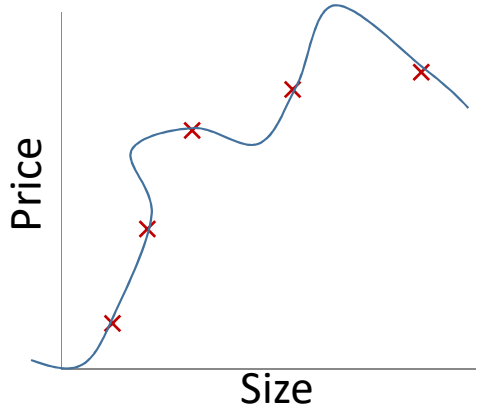
d = 1



$\theta_0 + \theta_1 x + \theta_2 x^2$

"Just right"

d = 2



$\theta_0 + \theta_1 x + \theta_2 x^2 + \theta_3 x^3 + \theta_4 x^4$

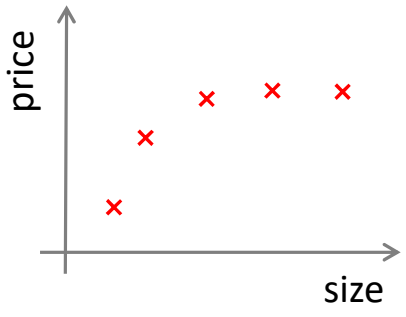
High variance (overfit)

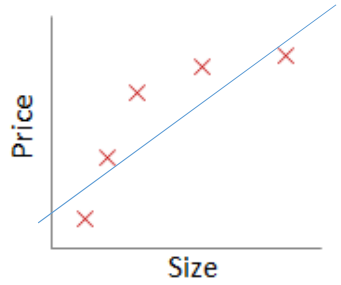
d = 4

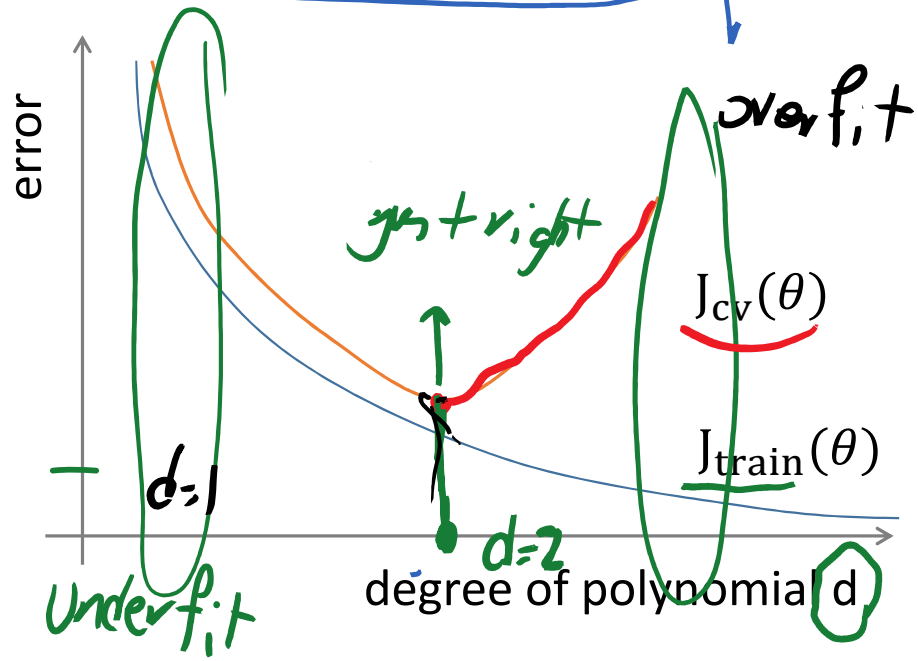
Bias/variance

Training error: $J_{train}(\theta) = \frac{1}{2m}\sum_{i=1}^{m}(h_\theta(x^{(i)}) - y^{(i)})^2$

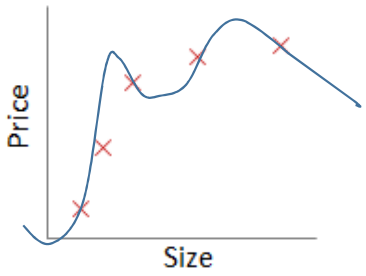
Cross validation error: $J_{cv}(\theta) = \frac{1}{2m_{cv}}\sum_{i=1}^{m_{cv}}(h_\theta(x_{cv}^{(i)}) - y_{cv}^{(i)})^2$





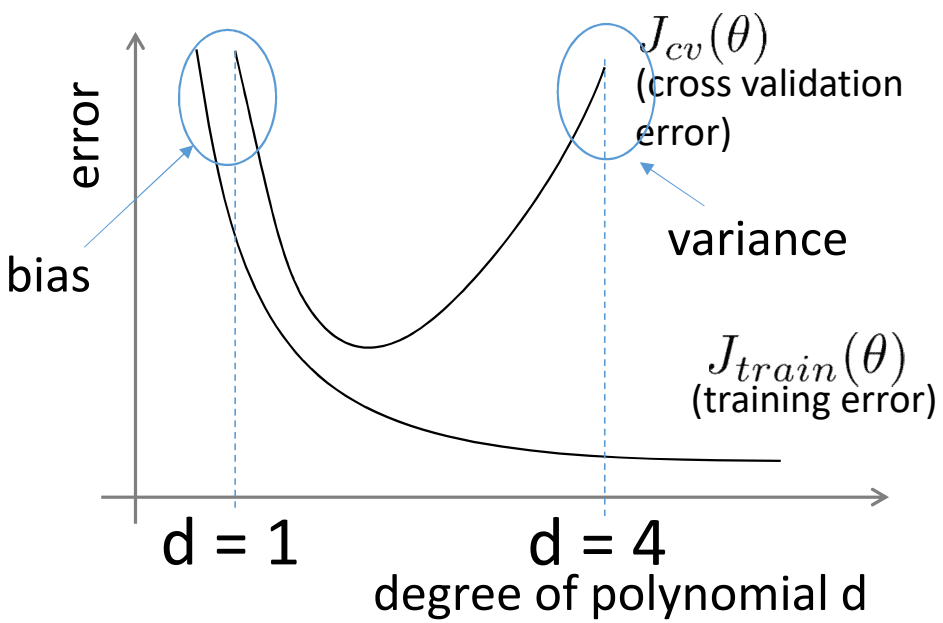


double descent Phenomena *



Diagnosing bias vs. variance

Suppose your learning algorithm is performing less well than you were hoping. ($J_{cv}(\theta)$ or $J_{test}(\theta)$ is high.) Is it a bias problem or a variance problem?



Bias (underfit):

$J_{train}(\theta)$ Will be high

$J_{cv}(\theta) \approx J_{train}(\theta)$

Variance (overfit):

$J_{train}(\theta)$ Will be low

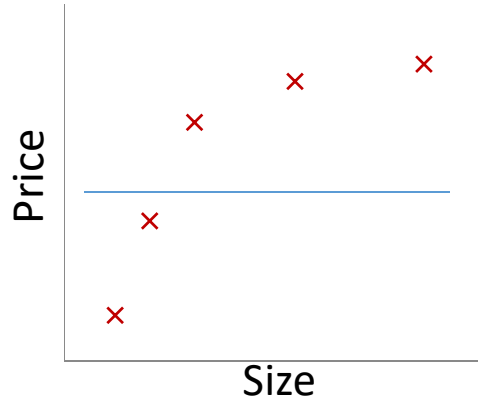
$J_{cv}(\theta) \gg J_{train}(\theta)$

Linear regression with regularization

Model: $h_\theta(x) = \theta_0 + \theta_1 x + \theta_2 x^2 + \theta_3 x^3 + \theta_4 x^4$

$$J(\theta) = \frac{1}{2m}\sum_{i=1}^{m}(h_\theta(x^{(i)}) - y^{(i)})^2 + \frac{\lambda}{2m}\sum_{j=1}^{n}\theta_j^2$$

پایداری

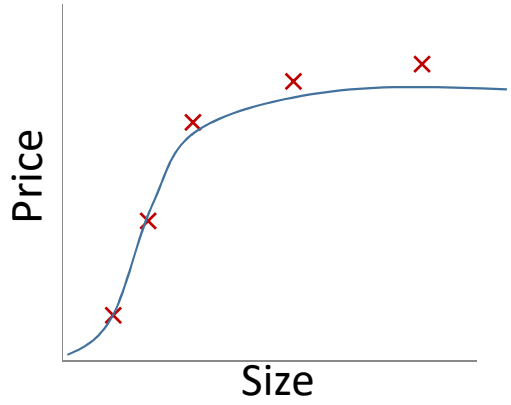


Large λ

High bias (underfit)

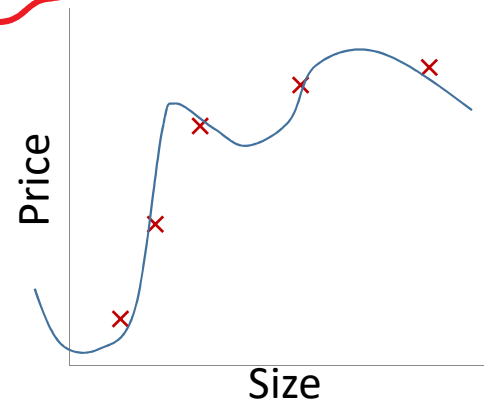
$\lambda = 10000.\ \theta_1 \approx 0, \theta_2 \approx 0, \ldots$

$h_\theta(x) \approx \theta_0$



Intermediate λ

"Just right"



Small λ

High variance (overfit)

Choosing the regularization hyperparameter λ

$$h_\theta(x) = \theta_0 + \theta_1 x + \theta_2 x^2 + \theta_3 x^3 + \theta_4 x^4$$

$$J(\theta) = \frac{1}{2m}\sum_{i=1}^{m}(h_\theta(x^{(i)}) - y^{(i)})^2 + \frac{\lambda}{2m}\sum_{j=1}^{m}\theta_j^2$$

$$J_{train}(\theta) = \frac{1}{2m}\sum_{i=1}^{m}(h_\theta(x^{(i)}) - y^{(i)})^2$$

$$J_{cv}(\theta) = \frac{1}{2m_{cv}}\sum_{i=1}^{m_{cv}}(h_\theta(x_{cv}^{(i)}) - y_{cv}^{(i)})^2$$

$$J_{test}(\theta) = \frac{1}{2m_{test}}\sum_{i=1}^{m_{test}}(h_\theta(x_{test}^{(i)}) - y_{test}^{(i)})^2$$

$\lambda = 0 \Rightarrow \text{argmin}\, J_{train} \rightarrow \theta^{(0)} \rightarrow J_{cv}(\theta^{(0)})$

$\lambda = 0.01$

$\lambda = 0.02$

$\lambda = 10 \Rightarrow \text{argmin}\, J_{train} \rightarrow \theta^{(10)} \Rightarrow J_{cv}(\theta^{(10)})$

$\lambda_{opt} = \text{argmin}\, J_{cv}(\theta^{(\lambda)})$

Choosing the regularization parameter λ

Model: $h_\theta(x) = \theta_0 + \theta_1 x + \theta_2 x^2 + \theta_3 x^3 + \theta_4 x^4$

$$J(\theta) = \frac{1}{2m}\sum_{i=1}^{m}(h_\theta(x^{(i)}) - y^{(i)})^2 + \frac{\lambda}{2m}\sum_{j=1}^{m}\theta_j^2$$

1. Try $\lambda = 0$
2. Try $\lambda = 0.01$
3. Try $\lambda = 0.02$
4. Try $\lambda = 0.04$
5. Try $\lambda = 0.08$

$\vdots$

12. Try $\lambda = 10$

$\Rightarrow J_{cv}(\theta^{(\lambda)})$

$\lambda_{opt} = \arg\min J_{cv}(\theta^{(\lambda)})$

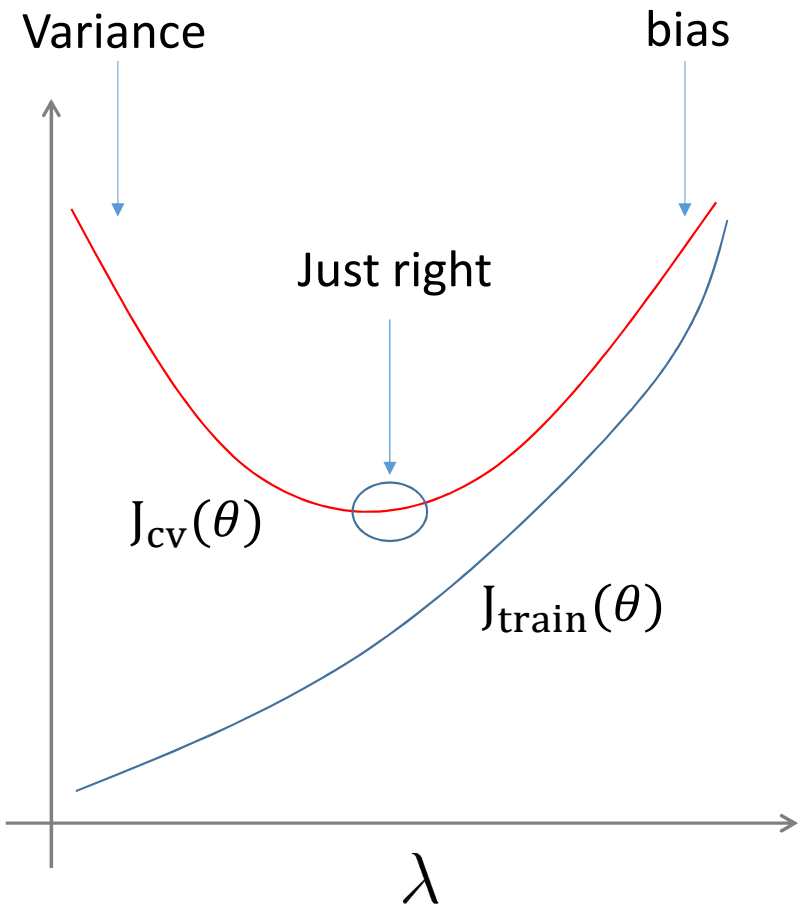
$\lambda_{opt} = 0.08$ ← Pick (say) $\theta^{(5)}$. Test error: $J_{test}(\theta^{(5)})$

Bias/variance as a function of the regularization parameter λ

$$J(\theta) = \frac{1}{2m}\sum_{i=1}^{m}(h_\theta(x^{(i)}) - y^{(i)})^2 + \frac{\lambda}{2m}\sum_{j=1}^{m}\theta_j^2$$

$$J_{train}(\theta) = \frac{1}{2m}\sum_{i=1}^{m}(h_\theta(x^{(i)}) - y^{(i)})^2$$

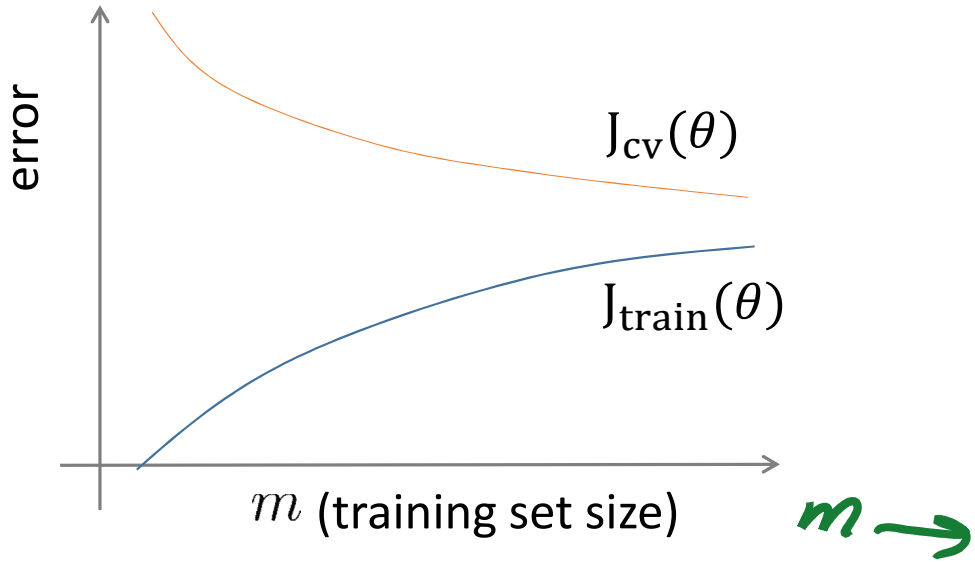
$$J_{cv}(\theta) = \frac{1}{2m_{cv}}\sum_{i=1}^{m_{cv}}(h_\theta(x_{cv}^{(i)}) - y_{cv}^{(i)})^2$$



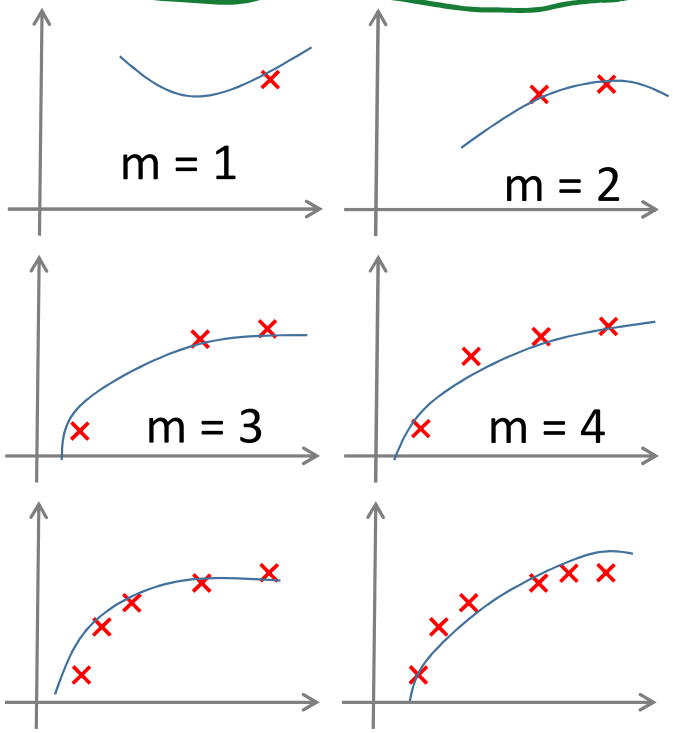
Learning curves

$$J_{train}(\theta) = \frac{1}{2m}\sum_{i=1}^{m}(h_\theta(x^{(i)}) - y^{(i)})^2$$

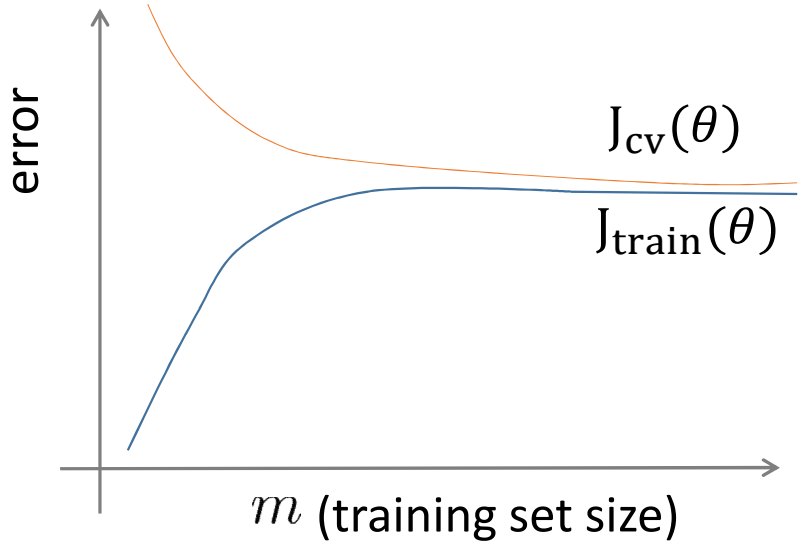
$$J_{cv}(\theta) = \frac{1}{2m_{cv}}\sum_{i=1}^{m_{cv}}(h_\theta(x_{cv}^{(i)}) - y_{cv}^{(i)})^2$$

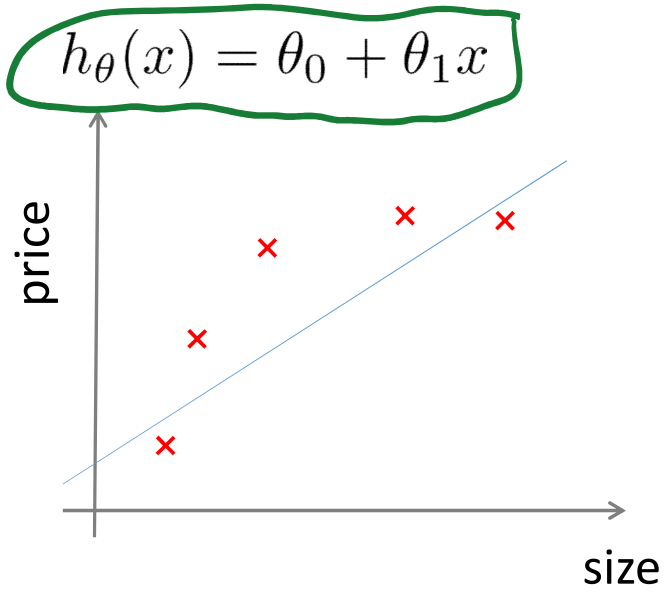


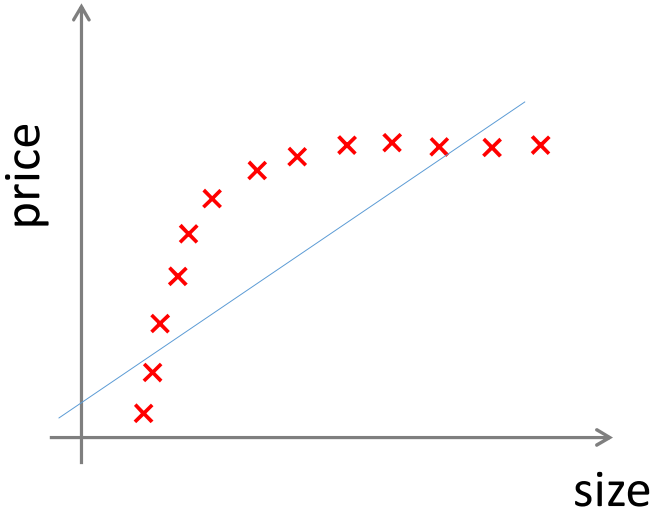
$$h_\theta(x) = \theta_0 + \theta_1 x + \theta_2 x^2$$



High bias

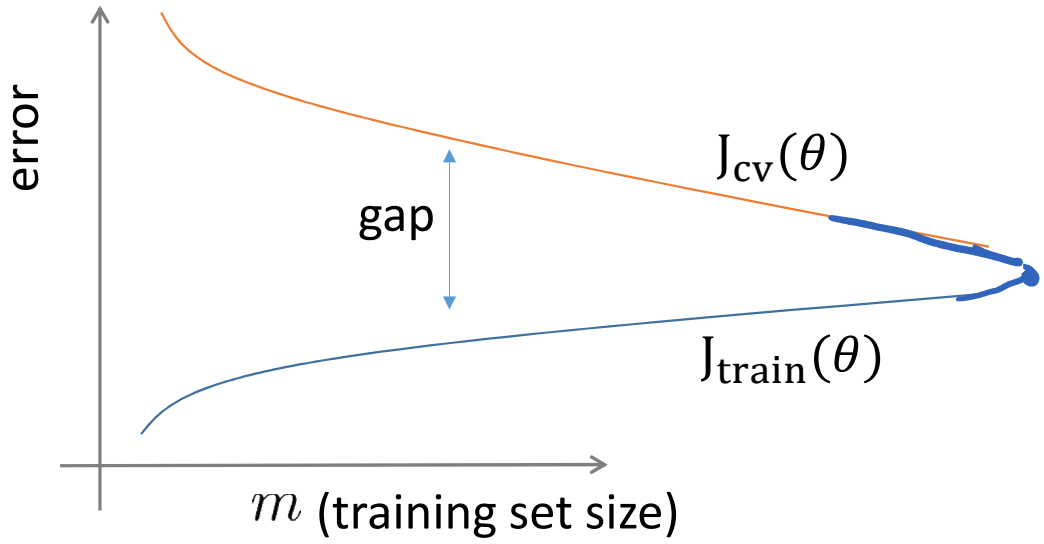






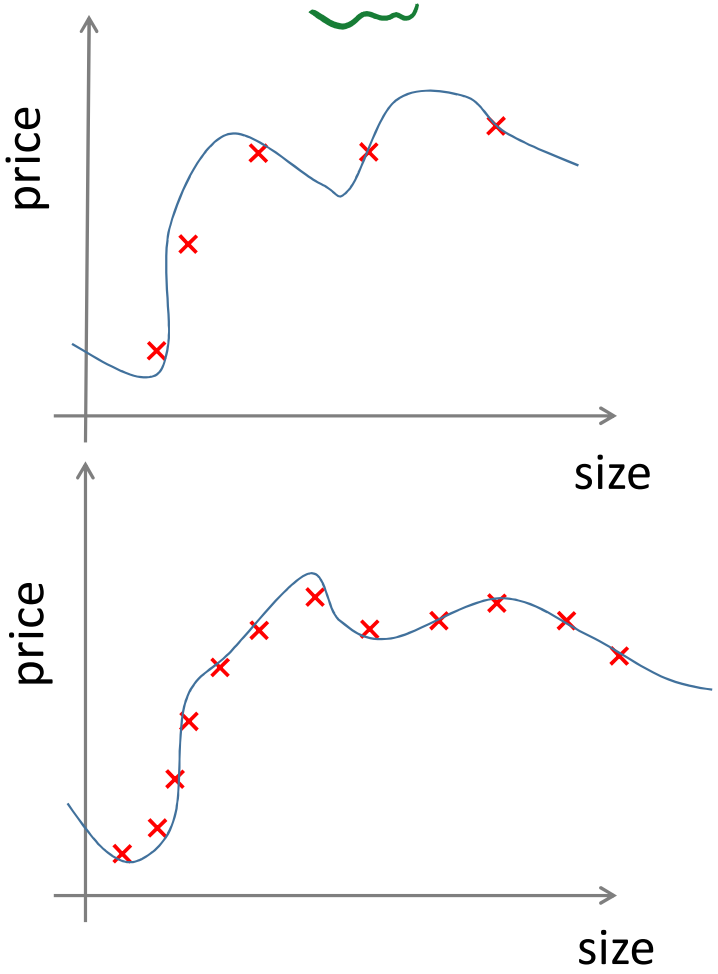
If a learning algorithm is suffering from high bias, getting more training data will not (by itself) help much.

High variance



$h_\theta(x) = \theta_0 + \theta_1 x + \cdots + \theta_{100} x^{100}$

(and small λ)



If a learning algorithm is suffering from high variance, getting more training data is likely to help.

$m \uparrow \rightarrow J_{cv} \downarrow$

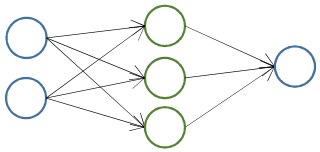
Debugging a learning algorithm:

Suppose you have implemented regularized linear regression to predict housing prices. However, when you test your hypothesis in a new set of houses, you find that it makes unacceptably large errors in its prediction. What should you try next?

- Get more training examples → fixes high variance
- Try smaller sets of features → fixes high variance
- Try getting additional features → fixes high bias
- Try adding polynomial features $(x_1^2, x_2^2, x_1x_2, \text{etc})$ → fixes high bias
- Try decreasing λ → fixes high bias
- Try increasing λ → fixes high variance

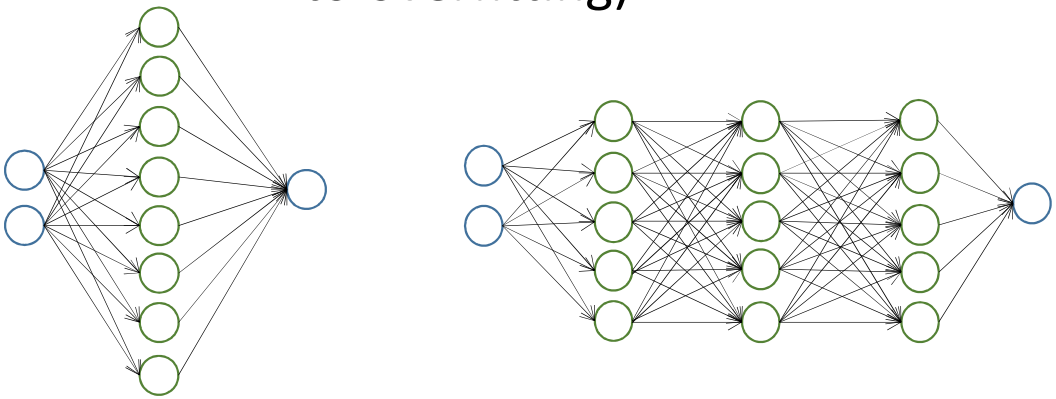
Neural networks and overfitting

"Small" neural network
(fewer parameters; more prone to underfitting)



Computationally cheaper

"Large" neural network
(more parameters; more prone to overfitting)



Computationally more expensive.

Use regularization (λ) to address overfitting.

برای انتخاب تعداد لایه ها یا تعداد نورون ها در هر لایه می توان از مجموعه validation استفاده کرد.

Error Metrics for Skewed Classes

Overall accuracy:

$$\frac{\text{TruePositive} + \text{TrueNegative}}{\text{TotalSample}\ (m)}$$

avg accuracy:

$$C1 = \frac{Ture\ for\ class\ 1}{TotalSample\ for\ class\ 1}$$

$$C2 = \frac{Ture\ for\ class\ 2}{TotalSample\ for\ class\ 2}$$

$$\frac{C1 + C2}{2}$$

skewed classes:

$h_\theta(x)$ $y \in \{0, 1\}$

0: سالم 1: سرطانی

طبقه‌بندی باینری: سالم / سرطانی

اگر فقط 0.5٪ از داده‌ها آموزشی سرطانی باشد

(=) 0.5٪ خطای خوبیست

1٪ خطا — 99٪ دقت

Prediction

y = prediction(x)

y = 0 ; X بدون توجه

return

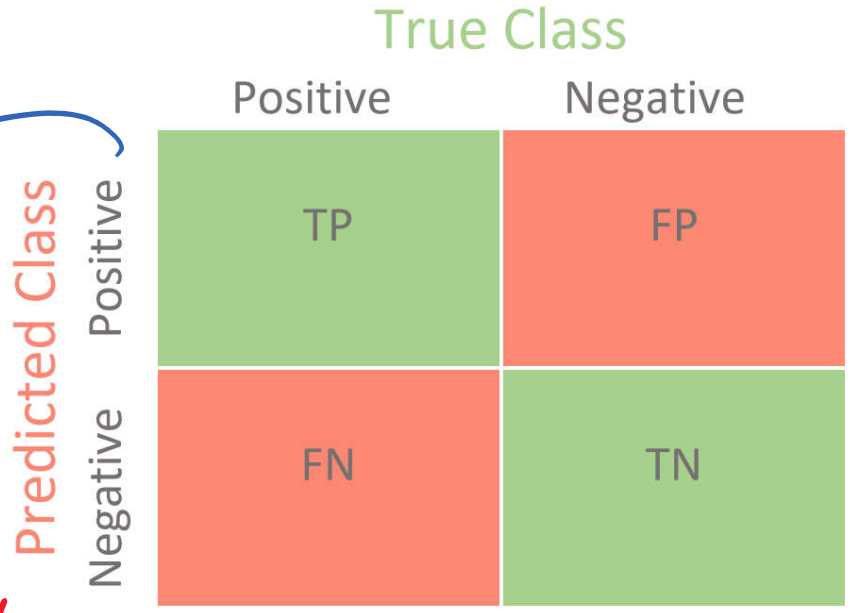
0.5٪: خطا

Overall acc: 99.5٪

Avg acc: $\frac{100 + 0}{2} = 50\%$

Metrics

Confusion matrix



$$\text{Recall} = \frac{TP}{TP+FN}$$

$$\text{Precision} = \frac{TP}{TP+FP}$$

$$\text{Accuracy} = \frac{TP+TN}{TP+FP+FN+TN}$$

$$\text{F1 Score} = 2 \times \frac{\text{recall} \times \text{precision}}{\text{recall} + \text{precision}}$$

log. Reg

$0 \leq h_\theta(x) \leq 1$

$h_\theta(x) = p(y=1|x)$

$h_\theta(x) \geq 0.5 \Rightarrow y = 1$

$h_\theta(x) < 0.5 \Rightarrow y = 0$

فرض کنید می خواهیم در صورت احتمال زیاد $y=1$ را اعلام کنیم (بیمار سرطان دارد)

$th = 0.7$ $\quad h_\theta \geq 0.7 \longrightarrow y = 1$ $\quad$ precision ↑

$h_\theta < 0.7 \longrightarrow y = 0$ $\quad$ recall ↓

فرض کنید که می خواهیم احتمال کم که هیچ بیمار سرطانی به اشتباه سالم اعلام نشود

$th = 0.3$ (avoid false negative)

precision ↓ $\quad$ recall ↑

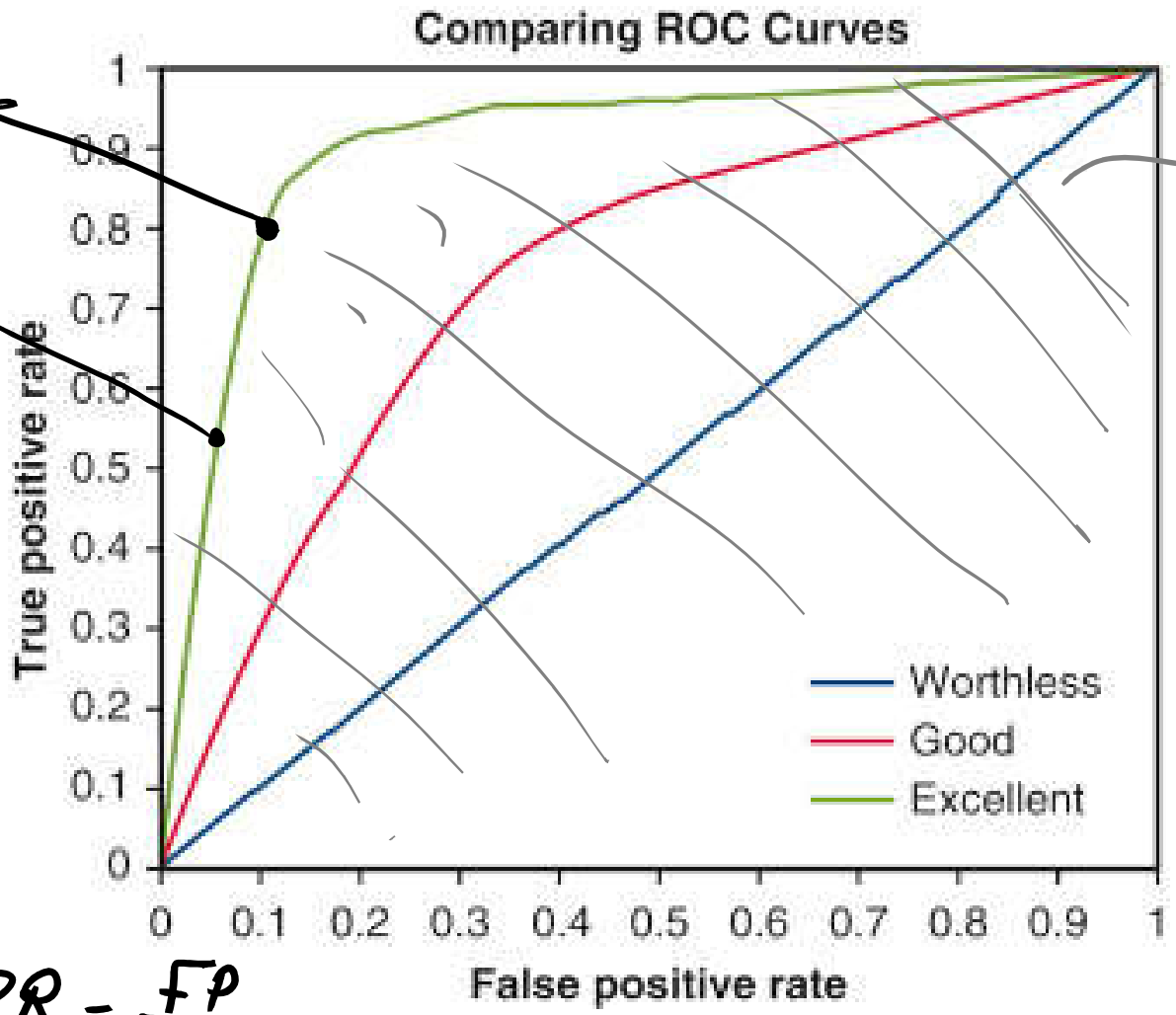
	Precision	Recall
Algorithm 1	0/5	0/4
... 2	0/7	0/1
... 3	0/02	1

→ بیشترین Fscore را دارد.

$\frac{P+R}{2}$

الزوما P یا R بزرگ باشد هم این مقدار بزرگ است.

F_1 Score (F) : $\frac{2PR}{P+R}$ ، $F < 1$



th = 0.3

th = 0.7

$TPR = \frac{TP}{TP + FN}$

$FPR = \frac{FP}{TN + FP}$

AUC : Area Under Curve

$0 \leq AUC \leq 1$

Receiver Operation Characteristic (ROC)

ROC، یک TPR را در مقابل FPR برای th های مختلف نشان می دهد.

(Binary classification)

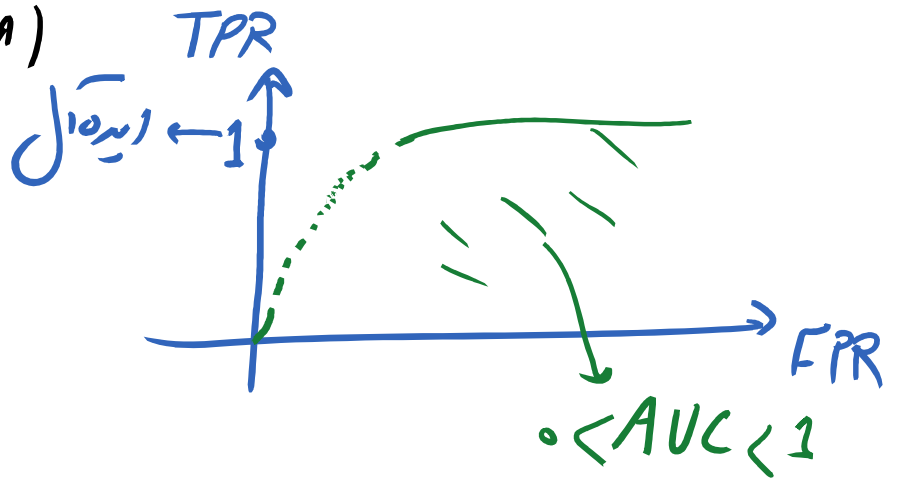
TPR: sensitivity — چه بخشی از بیماران واقعی تشخیص داده شده.

$$\frac{TP}{TP+FN}$$

FPR: 1_specifity — چه بخشی از سالم های واقعی تشخیص داده نشده.

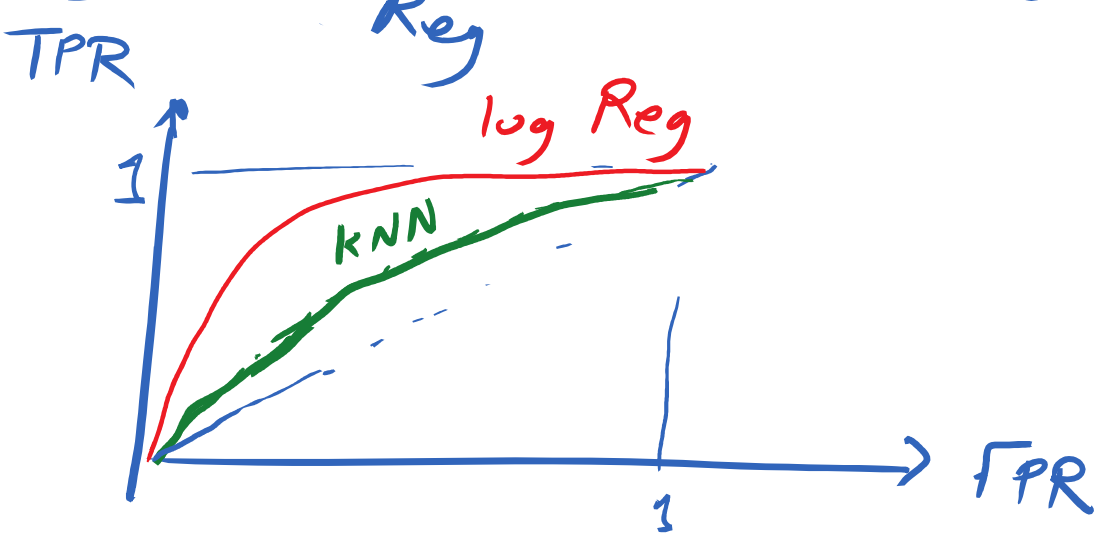
$$\frac{FP}{TN+FP}$$

$$\frac{TN}{TN+FP}$$



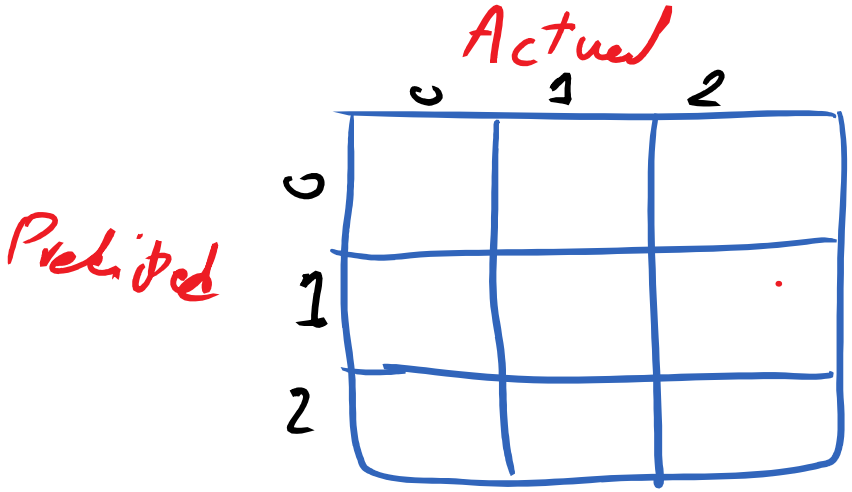
Predicted \ Actual	Pos	Neg
Pos	TP	FP
Neg	FN	TN

فعلاً مقایسه روش KNN و Log Reg برای یک دیتاست خاص (مسئله طبقه‌بندی باینری)



در کلاس حل تمرین با sklearn برای یک نمونه دیتاست پیاده سازی شود. (ROC رسم شود)

Confusion matrix



one_vs_all : برای بیش از یک کلاس

